

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO2: *Apply Dynamic Programming methods to solve reinforcement learning problems and derive optimal policies for decision-making tasks. (BL2, BL3)*

Assessment Tool Weightage: 30%

Dr G.A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Application - Based Questions

Duration: 2Hrs

1. Design and develop a Reinforcement Learning framework a ride-sharing company uses Reinforcement Learning to match drivers with passengers efficiently. Apply RL concepts to define the state space, action space, and reward function to minimize waiting time and improve service quality. **(5 Marks)**
2. An e-learning platform uses Reinforcement Learning to recommend personalized study materials. Recall how exploration and exploitation strategies help balance familiar and new content recommendations. **(5 Marks)**
3. Design and develop a Reinforcement Learning framework a robotic vacuum cleaner uses Reinforcement Learning to clean efficiently while avoiding obstacles. Apply the concept of policy and explain how the value function improves performance over time. **(5 Marks)**
4. A cryptocurrency trading system uses Reinforcement Learning to decide buy, sell, or hold actions. Outline the challenges in designing reward functions and explain how poor reward design can affect performance. **(5 Marks)**
5. Develop a smart grid system uses Reinforcement Learning to manage electricity distribution. Create an RL model by defining state space, action space, and reward mechanism. **(5 Marks)**

6. A healthcare monitoring system uses Reinforcement Learning to adjust medication dosages. Apply RL concepts to define states, actions, and rewards, and explain how learning improves outcomes. **(5 Marks)**

7. Design a model drone delivery system uses Reinforcement Learning for navigation. Recall how exploration and exploitation strategies influence route optimization. **(5 Marks)**

8. **Design and develop** a Reinforcement Learning framework a manufacturing robot uses Reinforcement Learning to optimize assembly operations. Apply the concept of policy and explain how value functions help improve efficiency. **(5 Marks)**

9. An online advertising system uses Reinforcement Learning to maximize click-through rates. Outline reward design challenges and explain their impact on system performance. **(5 Marks)**

10. A smart irrigation system uses Reinforcement Learning to optimize water usage. Create an RL framework by defining states, actions, and reward function. **(5 Marks)**

Code of Conduct:

*I T.chaitra(192425217) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: T.chaitra

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

1. Ride-Sharing Company using Reinforcement Learning

Theory

A ride-sharing company uses RL to assign drivers to passengers efficiently.

State Space (S):

- Driver location
- Passenger location
- Traffic condition
- Number of available drivers

Action Space (A):

- Assign Driver 1
- Assign Driver 2
- Wait for another driver

Reward Function (R):

- +10 → Passenger picked quickly
- +5 → Short travel distance
- -10 → Long waiting time
- -5 → Wrong assignment

The RL agent learns to reduce passenger waiting time and improve service quality.

Simple Python Code

```
import random
```

```
states = ["Near", "Far"]
```

```
actions = ["Assign Driver", "Wait"]
```

```
state = random.choice(states)
```

```
action = random.choice(actions)
```

```
if state == "Near" and action == "Assign Driver":
```

```
    reward = 10
```

```
else:
```

```
    reward = -5
```

```
print("State:", state)
```

```
print("Action:", action)
```

```
print("Reward:", reward)
```

Output:



```
Out[1]: Near
Out[2]: Assign Driver
Out[3]: 10
```

2. E-Learning Platform (Exploration vs Exploitation) Theory

RL recommends study materials.

Exploration

- Recommends new topics.
- Helps discover better learning resources.

Exploitation

- Recommends materials the student already likes.
- Improves learning efficiency.

Balancing both gives personalized recommendations.

Simple Python Code

```
import random
```

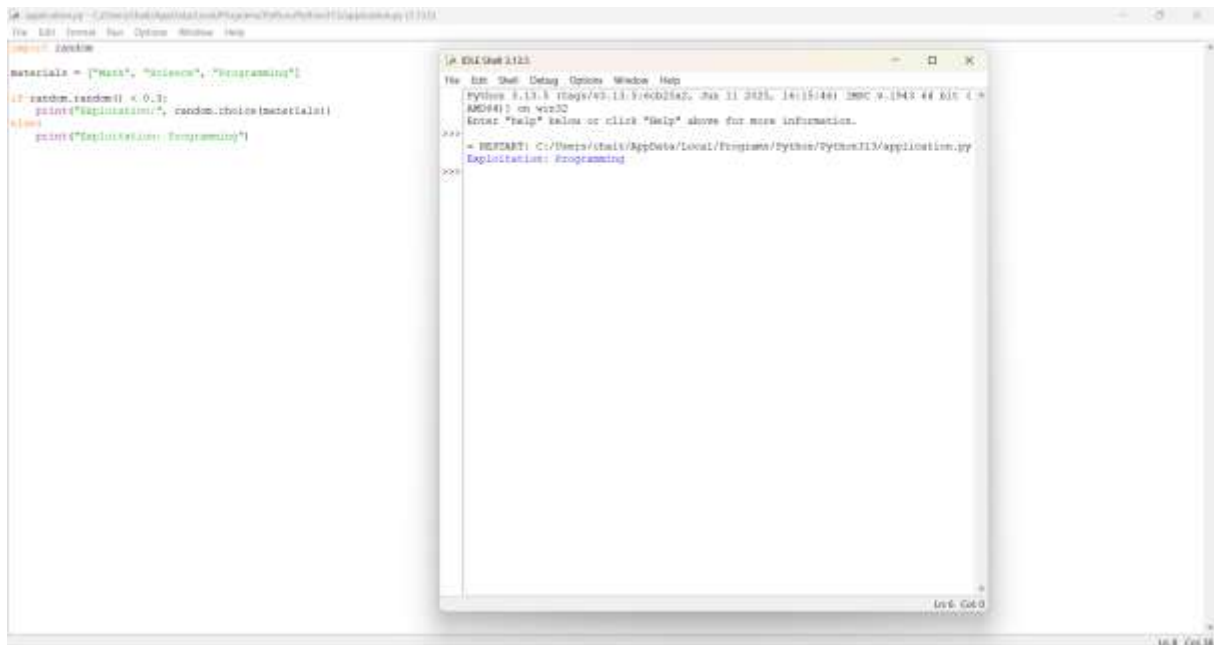
```
materials = ["Math", "Science", "Programming"]
```

```
if random.random() < 0.3:
```

```
    print("Exploration:", random.choice(materials))
```

```
else:
```

```
    print("Exploitation: Programming")
```



3. Robotic Vacuum Cleaner

Theory

The robot learns to clean efficiently.

States

- Clean room
- Dirty room
- Obstacle detected

Actions

- Move Forward
- Left
- Right
- Clean

Policy

A policy tells the robot what action to take in each state.

Value Function

The value function estimates future rewards.

Higher values help choose better cleaning paths.

Simple Python Code

```
state = "Dirty"
```

```
if state == "Dirty":
```

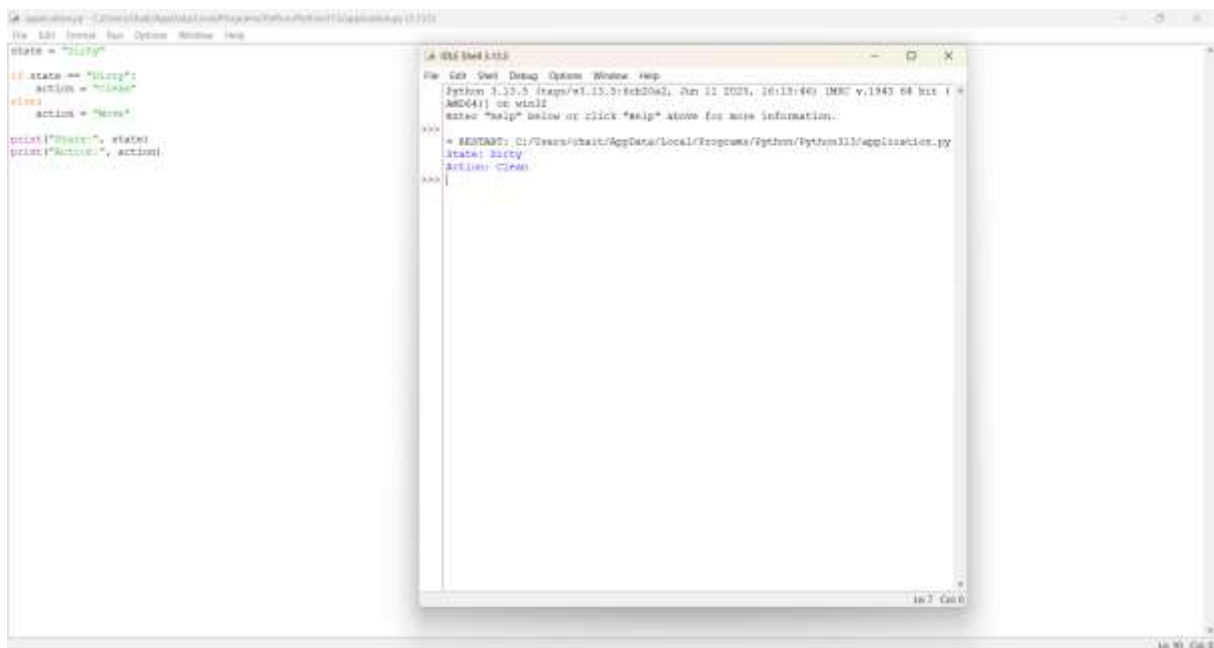
```
    action = "Clean"
```

```
else:
```

```
    action = "Move"
```

```
print("State:", state)
```

```
print("Action:", action)
```



```
state = "Dirty"

if state == "Dirty":
    action = "Clean"
else:
    action = "Move"

print("State:", state)
print("Action:", action)
```

```
Python 3.10.5 (tags/v3.10.5:8cd30e2, Jun 11 2023, 10:13:40) [AMD64] on win32
Enter "help" below or click "help" above for more information.

>>>
= RESTART: C:\Users\thait\AppData\Local\Programs\Python\Python310\python.exe
state: Dirty
Action: Clean
>>>
```

4. Cryptocurrency Trading System

Theory

Actions:

- Buy
- Sell
- Hold

Reward Design Challenges

- Market is highly unpredictable.
- Profit may come after several trades.
- Wrong rewards encourage risky trading.

Poor Reward Design

- Too much buying
- Too much selling
- Large financial losses

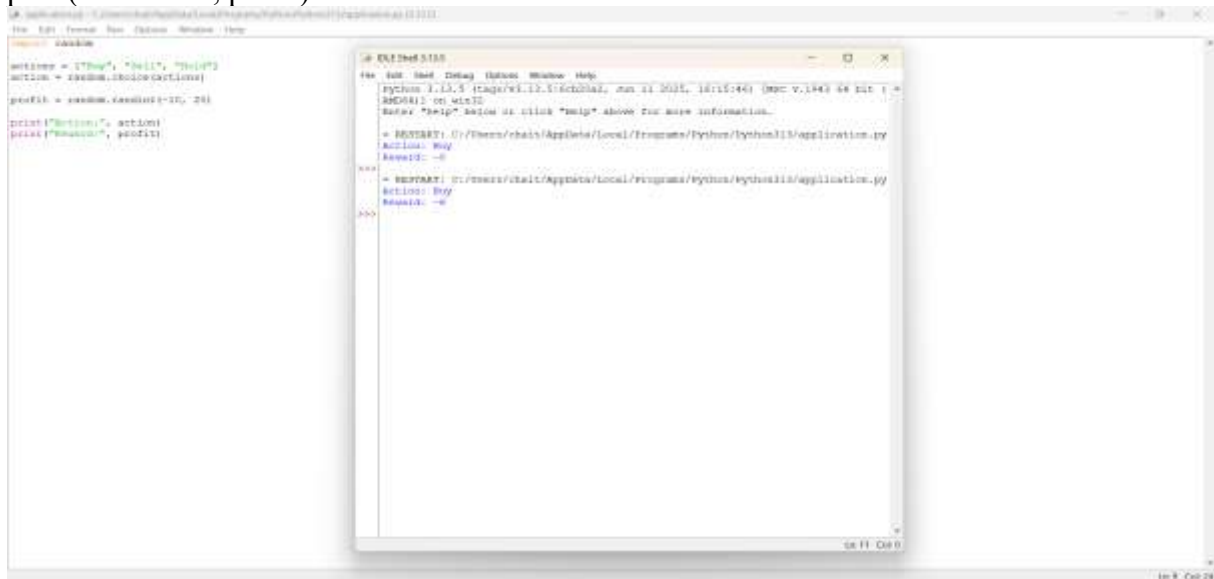
Simple Python Code

```
import random
```

```
actions = ["Buy", "Sell", "Hold"]
action = random.choice(actions)
```

```
profit = random.randint(-10, 20)
```

```
print("Action:", action)
print("Reward:", profit)
```



5. Smart Grid System

Theory

RL distributes electricity efficiently.

States

- Low demand
- Medium demand
- High demand

Actions

- Increase supply
- Reduce supply
- Maintain supply

Reward

+10 → Balanced electricity

-10 → Power shortage

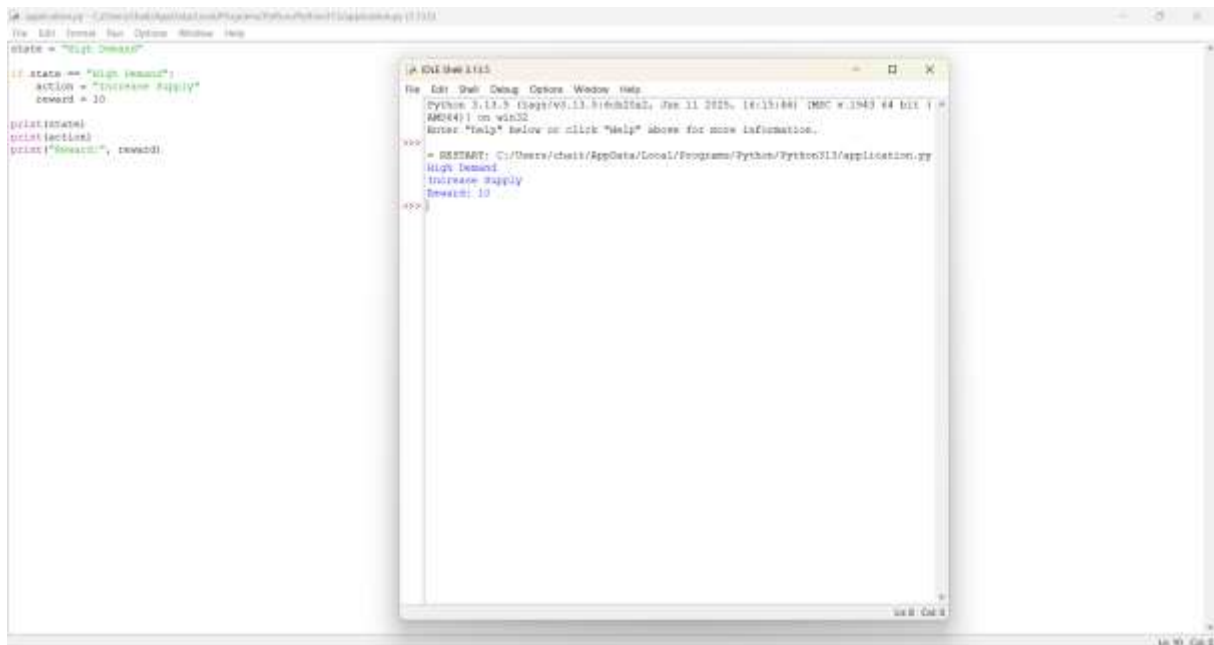
-5 → Power waste

Simple Python Code

```
state = "High Demand"
```

```
if state == "High Demand":
    action = "Increase Supply"
    reward = 10
```

```
print(state)
print(action)
print("Reward:", reward)
```



6. Healthcare Monitoring System

Theory

RL adjusts medicine dosage.

States

- Low BP
- Normal BP
- High BP

Actions

- Increase dosage
- Reduce dosage
- Keep same dosage

Reward

+10 → Healthy patient

-10 → Incorrect dosage

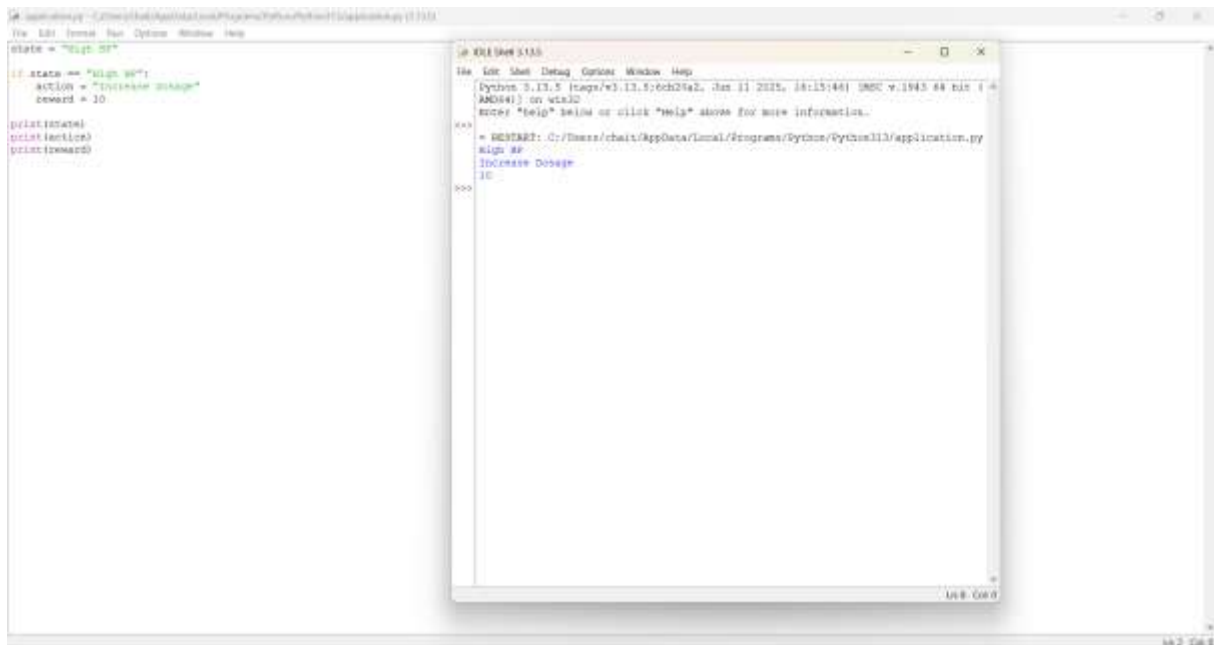
Learning improves treatment over time.

Simple Python Code

```
state = "High BP"
```

```
if state == "High BP":
    action = "Increase Dosage"
    reward = 10
```

```
print(state)
print(action)
print(reward)
```

7. Drone Delivery System

Theory

The drone learns the shortest delivery route.

Exploration

- Tries new routes.

Exploitation

- Uses the best-known route.

Balancing both reduces delivery time.

Simple Python Code

```
import random
```

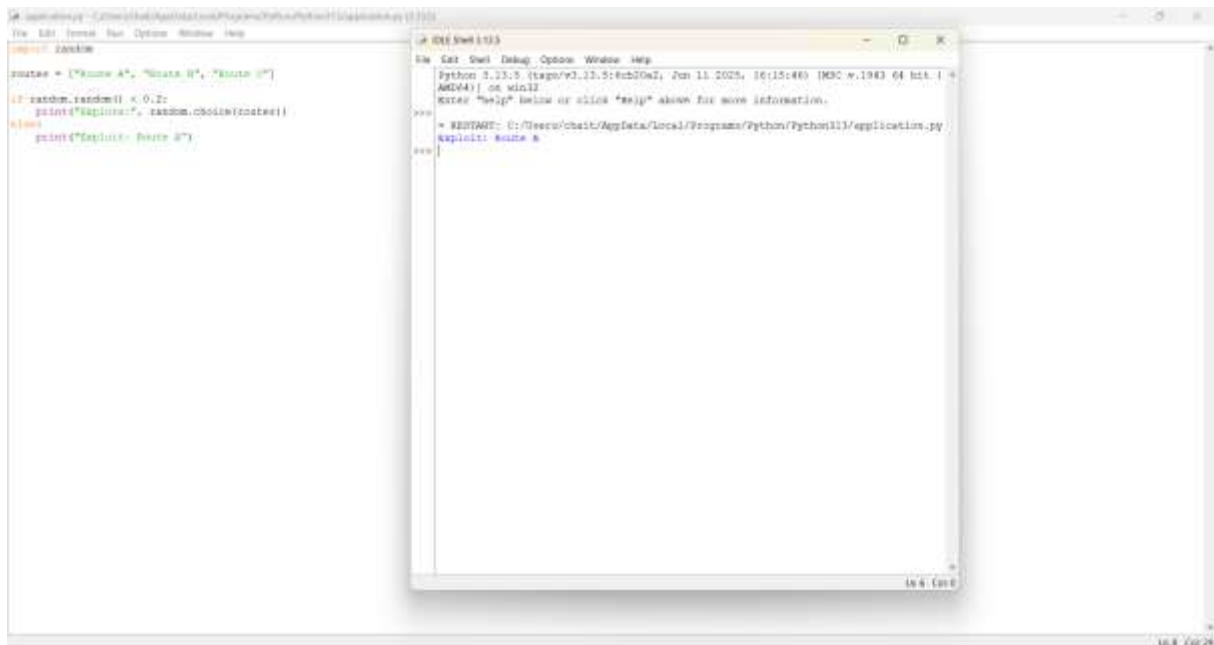
```
routes = ["Route A", "Route B", "Route C"]
```

```
if random.random() < 0.2:
```

```
    print("Explore:", random.choice(routes))
```

```
else:
```

```
    print("Exploit: Route A")
```



8. Manufacturing Robot

Theory

The robot learns efficient assembly operations.

States

- Part Available
- Machine Busy
- Assembly Complete

Actions

- Pick Part
- Assemble
- Wait

Policy

Maps each state to the best action.

Value Function

Predicts future rewards.

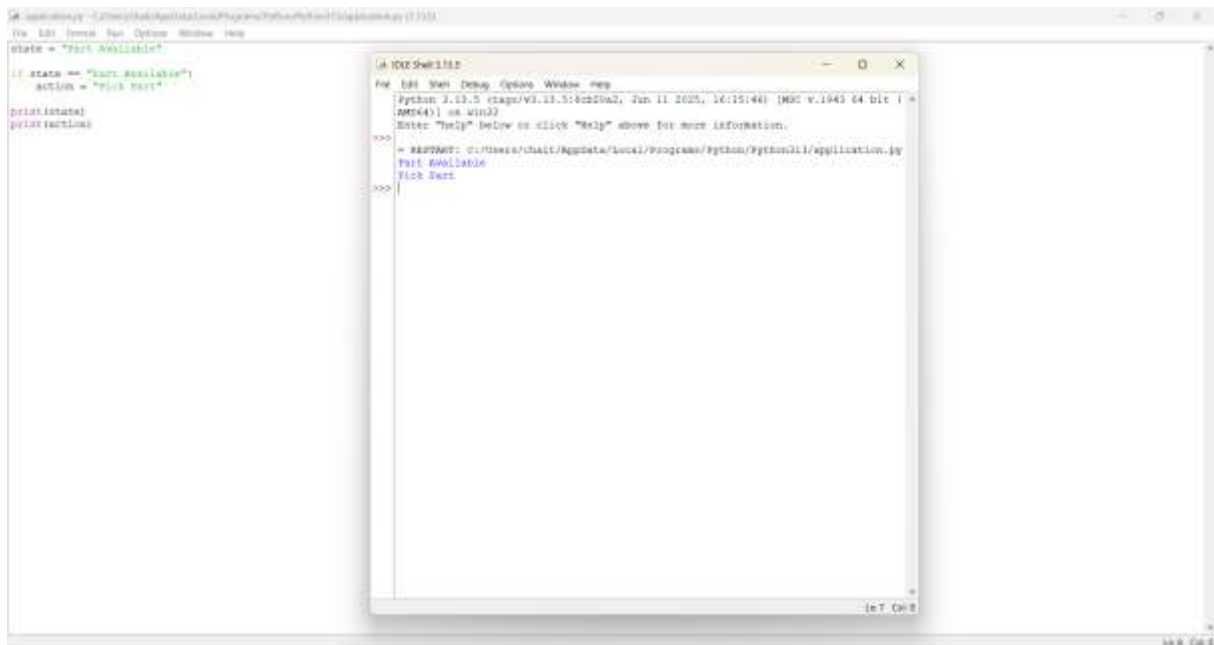
Higher values improve efficiency.

Simple Python Code

```
state = "Part Available"
```

```
if state == "Part Available":
    action = "Pick Part"
```

```
print(state)
print(action)
```



9. Online Advertising System

Theory

Actions:

- Show Ad A
- Show Ad B
- Show Ad C

Reward Challenges

- Delayed user clicks.
- Different user interests.
- Clicks do not always mean purchases.

Poor Reward Design

- Wrong advertisements.
- Low click-through rate.
- Poor customer experience.

Simple Python Code

```
import random
```

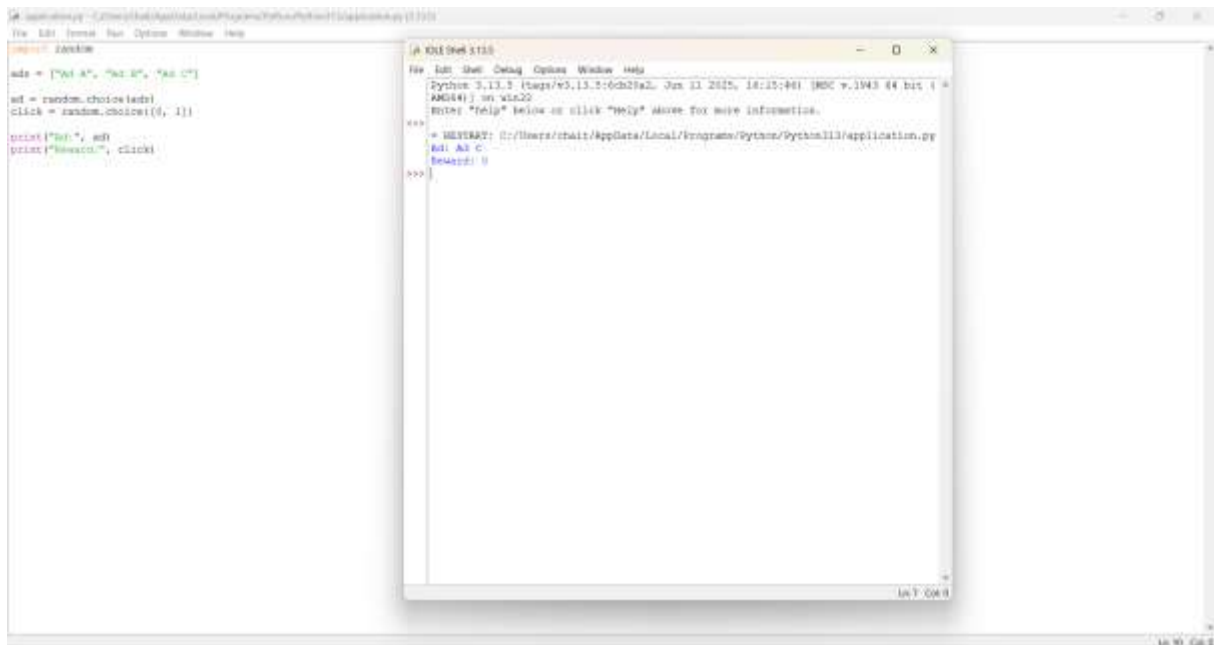
```
ads = ["Ad A", "Ad B", "Ad C"]
```

```
ad = random.choice(ads)
```

```
click = random.choice([0, 1])
```

```
print("Ad:", ad)
```

```
print("Reward:", click)
```



10. Smart Irrigation System

Theory

RL saves water while keeping crops healthy.

States

- Soil Moisture
- Weather
- Temperature

Actions

- Increase Water
- Reduce Water
- Stop Water

Reward

+10 → Healthy crops with less water

-10 → Water wastage

-5 → Dry soil

Simple Python Code

```
soil = "Dry"
```

```
if soil == "Dry":
```

```
    action = "Increase Water"
```

```
    reward = 10
```

```
print(soil)
```

```
print(action)
```

```
print(reward)
```

